

Capitolul 8

Interpolarea trigonometrică și TFR. Compresia

8.1 Transformata Fourier

Comanda MATLAB `fft` efectuează TFD cu o normalizare ușor diferită, astfel că $F_n x$ este calculat folosind expresia `fft(x)/sqrt(n)`. Comanda `ifft` este inversa lui `fft`. Prin urmare, $F_n^{-1} y$ se calculează folosind comanda MATLAB `ifft(y)*sqrt(n)`. Cu alte cuvinte, comenzile MATLAB `fft` și `ifft` sunt inverse una alteia, deși normalizarea lor diferă față de definiția dată în curs, care are avantajul că F_n și F_n^{-1} sunt matrici unitare.

8.2 Interpolarea trigonometrică

Vom încerca să scriem codul MATLAB pentru a implementa „Evaluarea eficientă a funcțiilor trigonometrice”. De fapt, vrem să implementăm

$$F_p^{-1} \sqrt{\frac{p}{n}} F_n x$$

folosind comenzile MATLAB `fft` și `ifft`, unde

$$F_p^{-1} = \sqrt{p} \cdot \text{ifft} \text{ și } F_n = \frac{1}{\sqrt{n}} \cdot \text{fft}.$$

Punerea celor două părți împreună corespunde următoarelor operații:

$$\sqrt{p} \cdot \text{ifft}_{[p]} \sqrt{\frac{p}{n}} \frac{1}{\sqrt{n}} \cdot \text{fft}_{[n]} = \frac{p}{n} \cdot \text{ifft}_{[p]} \cdot \text{fft}_{[n]}.$$

Bineînțeles, F_p^{-1} poate fi aplicată doar pentru un vector de lungime p , deci trebuie să plasăm coeficienții Fourier de gradul n într-un vector de lungime p înainte să facem inversarea. Programul `dftinterp.m` face acești pași.

```
function xp=dftinterp(inter,x,n,p)
c=inter(1);
d=inter(2);
t=c+(d-c)*(0:n-1)/n;
tp=c+(d-c)*(0:p-1)/p;
y=fft(x);
yp=zeros(p,1);
```

```

yp(1:n/2+1)=y(1:n/2+1);
yp(p-n/2+2:p)=y(n/2+2:n);
xp=real(fft(yp))*(p/n);
plot(t,x,'o',tp,xp)

```

Rulând această funcție

```
>> dftinterp([0, 1], [-2.2 -2.8 -6.1 -3.9 0.0 1.1 -0.6 -1.1],8,100)
```

de exemplu, produce punctele cu $p = 100$ reprezentate grafic în Figura 8.6 din curs fără a folosi explicit sinuși sau cosinuși. Trebuie făcute câteva comentarii despre acest cod. Scopul este de a aplica $\text{fft}_{[n]}$, urmată de $\text{ifft}_{[p]}$, și apoi de a înmulți cu p/n . După ce aplicăm fft celor n valori din x , coeficienții din vectorul y sunt mutați de la cele n frecvențe din $P_n(t)$ într-un vector yp care are p frecvențe, unde $p \geq n$. Sunt multe frecvențe înalte între cele p frecvențe, care nu sunt folosite de către P_n , ceea ce duce la coeficienți egali cu zero pentru acele frecvențe înalte, în pozițiile de la $n/2 + 2$ la $p/2 + 1$. Jumătatea superioară a intrărilor din yp constituie o recapitulare a jumătății inferioare, cu conjugate complexe în ordine inversă, conform ecuației (8.12) din curs. După ce TFD este inversată folosind comanda ifft , deși rezultatul teoretic este real, din punct de vedere computațional s-ar putea să existe o parte imaginară de modul mic, parțial datorită rotunjirii. Aceasta este eliminată prin aplicarea comenzii real .

8.3 TFR și procesarea semnalelor

Codul pentru interpolarea trigonometrică de tip cele mai mici pătrate este dat în următoarea funcție MATLAB:

```

function xp=dftfilter(inter,x,m,n,p)
c=inter(1);
d=inter(2);
t=c+(d-c)*(0:n-1)/n;
tp=c+(d-c)*(0:p-1)/p;
y=fft(x);
yp=zeros(p,1);
yp(1:m/2)=y(1:m/2);
yp(m/2+1)=real(y(m/2+1));
if(m<n)
    yp(p-m/2+1)=yp(m/2+1);
end
yp(p-m/2+2:p)=y(n-m/2+2:n);
xp=real(ifft(yp))*(p/n);
plot(t,x,'o',tp,xp)

```

Figura 8.7 din curs prezintă interpolările de tip cele mai mici pătrate, generate de comanda

```
>> dftfilter([0,1], [-2.2,-2.8,-6.1,-3.9,0.0,1.1,-0.6,-1.1],m,8,200)
```

pentru $m = 4$ și, respectiv, $m = 6$.

8.4 TCD bi-dimensională și compresia imaginilor

Definirea matricei TCD C în MATLAB poate fi făcută folosind următorul fragment de cod:

```
for i=1:n
    for j=1:n
        C(i,j)=cos((i-1)*(2*j-1)*pi/(2*n));
    end
end
C=sqrt(2/n)*C;
C(1,:)=C(1,:)/sqrt(2);
```

Ca o alternativă, dacă Signal Processing Toolbox din MATLAB este disponibil, TCD uni-dimensională a unui vector x poate fi calculată astfel:

```
>> y=dct(x);
```

Pentru a obține TCD-2D a unei matrice X , folosim ecuația (9.10) din curs, adică

```
>> Y=C*X*C'
```

Dacă funcția `dct` din MATLAB este disponibilă, comanda

```
>> Y=dct(dct(X'))'
```

calculează TCD-2D folosind două aplicări ale TCD-1D.

MATLAB importă valori în tonuri de gri sau color RGB pentru imagini date în formate standard. De exemplu, fiind dat un fișier de imagine în tonuri de gri `picture.jpg`, comanda

```
>> x = imread('picture.jpg');
```

pune o matrice de valori în tonuri de gri în variabila în dublă precizie x . Dacă fișierul JPEG este o imagine color, variabila va avea o a treia dimensiune pentru a reține fiecare dintre cele trei culori. Ne vom îndrepta atenția către imagini în tonuri de gri, deoarece extensia la imagini color este ușoară.

O matrice $m \times n$ de valori în tonuri de gri poate fi desenată de către MATLAB folosind comenzile

```
>> imshow(x);colormap(gray)
```

iar o matrice $m \times n \times 3$ de culori RGB este desenată folosind doar comanda `imshow(x)` singură. O formulă comună pentru convertirea unei imagini color RGB într-una în tonuri de gri este

$$X_{\text{gray}} = 0.2126R + 0.7152G + 0.0722B, \quad (8.1)$$

sau, în cod MATLAB,

```
>> x=double(x);
>> r=x(:,:,1);g=x(:,:,2);b=x(:,:,3);
>> xgray=0.2126*r+0.7152*g+0.0722*b;
>> xgray=uint8(xgray);
>> imshow(xgray);colormap(gray)
```

Observăm că am convertit tipul de date implicit din MATLAB `uint8`, adică numere întregi fără semn, în numere reale în dublă precizie, înainte să facem calculele. Este bine să convertim înapoi la tipul `uint8` înaintea desenării imaginii folosind `imshow`.

În MATLAB, matricea de cuantizare liniară poate fi definită prin

```
>> Q=p*8./hilb(8);
```

În continuare, vom prezenta seria completă de pași pentru compresia unei matrici de pixeli în MATLAB. Ieșirea comenzii MATLAB `imread` este o matrice $m \times n$ de întregi pe 8 biți pentru o fotografie în tonuri de gri, sau trei astfel de matrici pentru o poză color. (Cele trei matrici conțin informația pentru roșu, verde, și, respectiv, albastru.) Un întreg pe 8 biți este de tip `uint8`, pentru a-l distinge de un `double`, care necesită 64 de biți pentru a fi stocat. Comanda `double(x)` convertește numărul `uint8 x` în format `double`, iar comanda `uint8(x)` face operația inversă prin rotunjirea lui x la cel mai apropiat întreg între 0 și 255.

Următoarele patru comenzi fac conversia, centrarea, transformarea, și cuantizarea unei matrici pătratică $n \times n$ X de numere `uint8`, asemenea matricilor de pixeli de dimensiune 8×8 considerate în curs. Notăm cu C matricea $n \times n$ a TCD.

```
>> Xd=double(X);
>> Xc=Xd-128;
>> Y=C*Xc*C';
>> Yq=round(Y./Q);
```

În acest moment, matricea Yq rezultată poate fi stocată sau transmisă. Pentru a recupera imaginea trebuie să reluăm cei patru pași de mai sus în ordine inversă:

```
>> Ydq=Yq.*Q;
>> Xdq=C'*Ydq*C;
>> Xe=Xdq+128;
>> Xf=uint8(Xe);
```

După decuantizare, TCD inversă este aplicată, deplasamentul 128 este adunat înapoi, și formatul `double` este convertit înapoi într-o matrice Xf de întregi `uint8`.

Probleme

1. Găsiți funcția de interpolare trigonometrică de ordinul 8 $P_8(t)$ pentru următoarele date:

t	x
0	0
1/8	1
1/4	2
3/8	3
1/2	4
5/8	5
3/4	6
7/8	7

Reprezentați grafic punctele și $P_8(t)$.

- Găsiți funcția de interpolare trigonometrică de ordinul $n = 8$ pentru $f(t) = e^t$ în punctele egal depărtate $(j/8, f(j/8))$ pentru $j = 0, \dots, 7$. Reprezentați grafic $f(t)$, punctele, și funcția de interpolare.
- Găsiți funcțiile de aproximare trigonometrică de tip cele mai mici pătrate de ordinele $m = 2$ și 4 pentru următoarele puncte:

t	x
0	3
1/4	1
1/2	-3
3/4	0

Folosind `dftfilter.m`, reprezentați grafic punctele și funcțiile de aproximare, ca în Figura 8.7 din curs.

- Găsiți funcțiile de aproximare de tip cele mai mici pătrate de ordinele 4 , 6 , și 8 pentru următoarele puncte:

t	x
0	1
1/8	2
1/4	3
3/8	1
1/2	-1
5/8	-1
3/4	-3
7/8	0

Reprezentați grafic punctele și funcțiile de aproximare, ca în Figura 8.7 din curs.

- Reprezentați grafic datele de mai jos împreună cu aproximările TCD de tip cele mai mici pătrate de ordinele $m = 4$, 6 , și 8 .

t	x
0	3
1	5
2	-1
3	3
4	1
5	3
6	-2
7	4

- Găsiți TCD-2D a matricii de date X .

$$X = \begin{bmatrix} -1 & 1 & -1 & 1 \\ -2 & 2 & -2 & 2 \\ -3 & 3 & -3 & 3 \\ -4 & 4 & -4 & 4 \end{bmatrix}.$$

7. Folosind TCD-2D din Problema 6, aproximarea filtrată trece-jos de tip cele mai mici pătrate pentru X , luând toate valorile de transformare $Y_{kl} = 0$ pentru $k + l \geq 4$.
8. Găsiți un fișier de imagine în tonuri de gri, și folosiți comanda `imread` pentru a o importa în MATLAB. Decupați matricea rezultată astfel încât fiecare dimensiune este un multiplu de 8. Dacă este necesar, convertiți o imagine RGB în tonuri de gri folosind formula standard (8.1).
 - (a) Extrageți un bloc de pixeli de dimensiune 8×8 , de exemplu, folosind comanda MATLAB `xb=x(81:88,81:88)`. Afișați blocul folosind comanda `imshow`.
 - (b) Aplicați TCD-2D acestui bloc.
 - (c) Cuantizați folosind cuantizarea liniară cu $p = 1, 2$, și 4 . Afișați fiecare matrice Y_Q .
 - (d) Reconstituiți blocul folosind TCD-2D inversă, și comparați rezultatul cu blocul inițial. Folosiți comenzile MATLAB `colormap(gray)` și `imshow(X,[0 255])`.
 - (e) Repetați pașii (1)–(5) pentru toate blocurile 8×8 ale imaginii, și reconstituiți imaginea în fiecare caz.
9. Repetați pașii din Problema 8, dar cuantizați folosind matricea sugerată de standardul JPEG, dată în ecuația (9.24) din curs, cu $p = 1$.